SAVEETHA SCHOOL OF ENGINEERING

C PROGRAMMING ASSIGNMENT

Comprehensive System Design and Implementation Challenge

Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Comprehensive System Design and Implementation Challenge – District Blood Bank Network
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	13 NOVEMBER 2026

TASK A3 — Design with Pointers (5 Marks)

Task / Problem Statement

Explain pointer arithmetic on the eight-element stock array, design a function that returns a pointer to the lowest stock, and design the command-line interface.

Aim

To use pointers to arrays and functions correctly and safely, and to read program input through argc / argv.

A3.1 – Pointer Operations (2 marks)

Given int stock[8]; int *p = stock; (p holds the address of stock[0]; an int is 4 bytes, so p + k is the base address plus 4·k bytes):

Expression	Meaning	Example (base 0x1000)
p + 3	A **pointer** to the 4th element, &stock[3]. Nothing is read; the address moves 3 × sizeof(int) = 12 bytes.	0x100C
*(p + 3)	The **value** stored at that address – the content of stock[3].	value of stock[3]
p[3]	Exactly the same as *(p + 3); the index notation is just shorthand.	value of stock[3]
*p++	++ binds tighter than *: the expression value is *p (that is stock[0]), **then** p is advanced by one element. p now holds &stock[1]. (It is not (*p)++, which would increase the stock value.)	p: 0x1000 → 0x1004

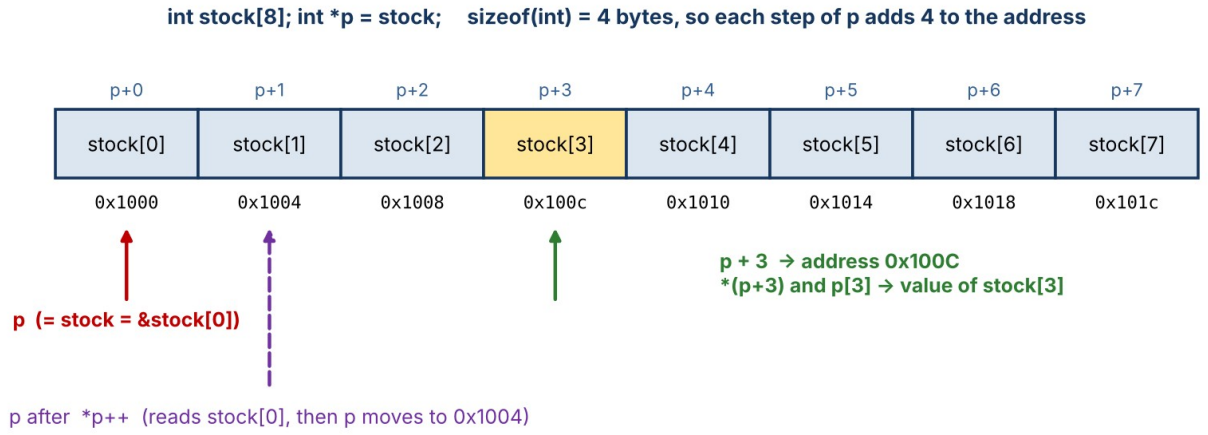


*Figure A3 – Memory diagram of stock[8], p, p + 3 and p after *p++.*

A3.2 – Pointers and Functions (2 marks)

```
ALGORITHM LowestStock(stock*, n)
  INPUT  : stock = address of first element, n > 0 = element count
  OUTPUT : address of the element holding the smallest units
BEGIN
  low <- &stock[0]
  FOR i <- 1 TO n - 1 DO
    IF stock[i].units < (*low).units THEN low <- &stock[i]
  END FOR
  RETURN low
END
```

```
Stock *lowest_stock(Stock *s, int n)   /* pointer INTO the caller's array */
{
   Stock *low = &s[0];
   for (int i = 1; i < n; i++)
      if (s[i].units < low->units) low = &s[i];
```

```
   return low;
}
/* caller:  lowest_stock(stock, 8)->units += 10;   updates the array directly */
```

Why returning a pointer to a local variable is wrong. A local variable lives in the function's stack frame, and that frame is destroyed when the function returns. The returned address then points to memory that will be reused by the next call, so the pointer is a **dangling pointer**: reading it gives garbage, writing corrupts other data (undefined behaviour). The design is safe because the pointer refers to the caller's array, which outlives the call. Other safe sources are malloced memory or a static variable.

A3.3 – Command-Line Interface (1 mark)

Command (argv[1] ...)	argc	Action	Checks
bloodbank stock	2	Print units of all 8 groups	No extra arguments
bloodbank find O+	3	List eligible donors of group O+, count and youngest	group in {A+,A-,B+,B-,AB+,AB-,O+,O-}
bloodbank issue A- 2	4	Reduce stock of A- by 2	valid group; units is a whole number 1..1000 (strtol); enough stock
bloodbank donate 10023	3	Record a donation by donor ID, add 1 unit	donor exists and is eligible
bloodbank register 10089 Vimal O+ 27 9812345670	7	Add a new donor	ID check digit, name < 32 chars, group, age 18–65
bloodbank lowest	2	Show the group with least stock	No extra arguments

Invalid arguments: the program never guesses. It (1) checks argc first, (2) compares argv[1] with the known command names, (3) validates every value, (4) prints a one-line message on **stderr**, plus the usage text for a missing or unknown command, and (5) ends with exit status **1** (0 means success). Numbers are converted with strtol and the end pointer is checked so abc or -5 is rejected.

Alternative Concept and Complexity Analysis – return the index

```
int lowest_index(const Stock *s, int n)
{ int low = 0; for (int i = 1; i < n; i++) if (s[i].units < s[low].units) low = i; return low; }

/* caller */   int i = lowest_index(stock, 8);   stock[i].units += 10;      /* pointer version:  lowest_stock(stock, 8)->units += 10; */
```

	Returns pointer	Returns index
Time	O(n) – one pass	O(n) – one pass
Space	O(1)	O(1)
Calling code	Shortest: lowest_stock(stock,8)->units += 10	Two steps: get index, then stock[i]; the index can also be printed or used to name the group (GROUP_NAMES[i])
Safety if the array is later reallocated	**Unsafe.** realloc may move the block; a stored Stock * still points to the old, freed block → dangling pointer	**Safe.** An index is a position, not an address – stock[i] is looked up again in the new block
Empty array	Can return NULL – caller must test	Needs a special value such as −1

Conclusion: both cost the same; use the pointer for an immediate update, and the index when the pointer must be kept across any operation that can reallocate the array.

Result

Pointer expressions were explained with a memory diagram, a safe pointer-returning function was designed and the command-line interface with its error handling was specified.